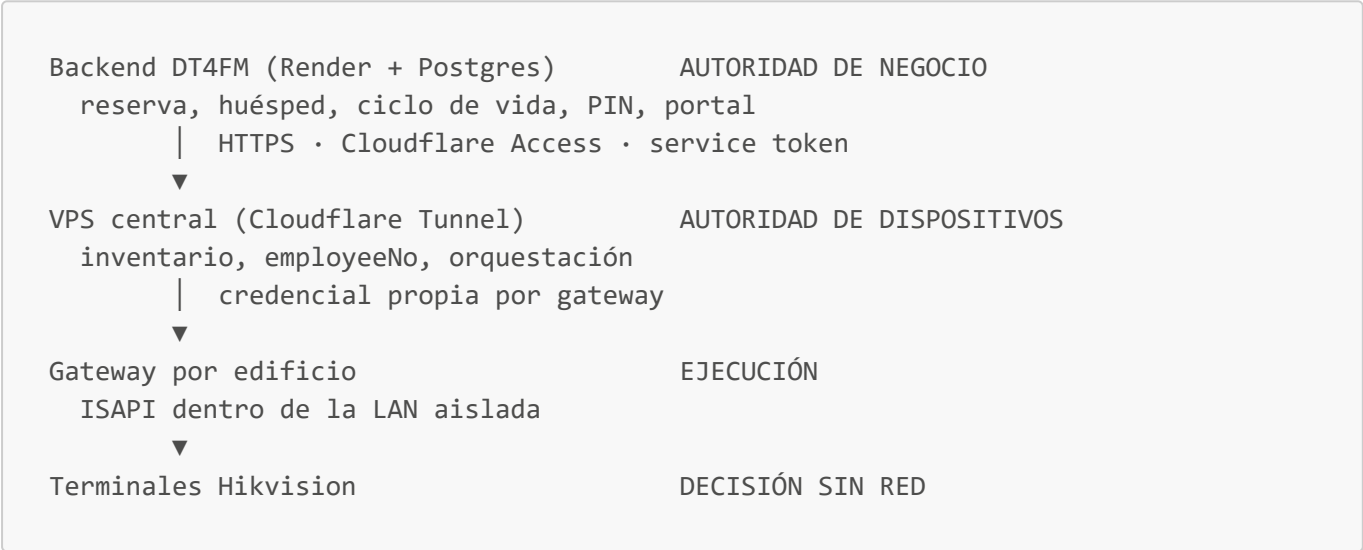


Control de accesos

Emite, sincroniza y revoca los PINes que abren las puertas de los edificios, y proyecta las reservas de Hostaway sobre las que se apoyará el portal del huésped.

Documento	Para qué
Este	Qué hace el módulo y cómo está construido
decisiones-arquitectura-y-seguridad.md	Por qué está construido así. D-01 a D-17, cerradas
guia-servidor-vps-accesos.md	Contrato para Ingeniería IoT: qué debe implementar la VPS

1. Las tres capas



El backend habla **solo** con la VPS: una URL y un service token. Nunca alcanza un terminal, y la VPS nunca llama al backend.

La VPS todavía no existe. Todo el módulo funciona hoy contra [access-iot.mock.ts](#), gobernado por `ACCESS_IOT_USE MOCK`.

2. Qué abre un PIN

No hay cerraduras por departamento. El control está solo en la **entrada peatonal** del edificio y en la **vehicular**; lo demás se abre con llave física. De ahí que `scope` sea `pedestrian` / `vehicular` / `both` y que `openmaint_unit_id` **no decida nada**: es contexto para soporte y para el portal.

Solo Inglaterra (ING, 3025058) y Pradera (PRA, 3019998) tienen puertas con PIN. Republica (564939) tiene reservas de Hostaway pero no lectores, así que una estancia sin credencial es el caso normal, no una anomalía. La cobertura se consulta a `GET /v1/buildings` y se cachea 10 minutos ([building-catalog.service.ts](#)): añadir hardware a un edificio nuevo es una operación del lado IoT, no un despliegue del backend.

Un edificio sin cobertura tiene **dos respuestas correctas**, y confundirlas ensucia el log o esconde un fallo real:

Origen	Comportamiento
Manual, <code>POST /access/credentials</code>	Error explícito. Pedir un PIN donde no hay lector es un error del operador
Automático, desde una reserva	No se emite, y ya. Nivel informativo. La estancia se crea igual

3. Modelo de datos

Dos tablas. [Migración 1788826000000](#).

`access_credential` — la autoridad

`subject_type` (`guest/tenant/employee`) + `subject_ref` identifican al dueño; para un huésped, `subject_ref` es el id de reserva de Hostaway. El PIN vive cifrado en `pin_ciphertext` y su huella HMAC en `pin_fingerprint`, que es lo que permite comprobar unicidad sin descifrar nada.

Dos ejes de estado que no hay que confundir:

- `status` — `pending` → `active` → `revoked` / `expired`. Es la **intención**.
- `sync_state` — `pending` / `synced` / `failed`. Es si esa intención llegó **al aparato**.

`sync_detail` (jsonb) guarda el resultado por dispositivo tal como lo devolvió la VPS. `failed` significa que la credencial existe en el sistema pero no en la puerta, o al revés: pide atención humana y es lo que cuenta `GET /access/health`.

Tres índices llevan el peso:

```
-- Unicidad del PIN por edificio, NO por (edificio, ámbito): un `both` y un
-- `pedestrian` con el mismo PIN no colisionan en el índice, pero sí en el
-- terminal peatonal, que es donde importa.
UNIQUE (building_id, pin_fingerprint)      WHERE status IN
('pending','active')

-- Impide la doble emisión cuando el webhook y el barrido coinciden.
UNIQUE (subject_type, subject_ref, scope)  WHERE status IN
('pending','active')

-- La cola de reintentos.
INDEX  (sync_state, updated_at)            WHERE sync_state <> 'synced'
```

`guest_stay` — la estancia

Se crea para **toda** reserva, haya cobertura o no: es el cimientito del portal, no un accesorio de los PINes. Guarda huésped, fechas, unidad y edificio resueltos, y la vigencia calculada. `token_version` y `guest_last_name_hash` están desde ya para no pagar una segunda migración cuando llegue el portal.

4. Ciclo de una credencial

```

issue()  —> pending/pending —> PUT a la VPS —> written —> active/synced
                                         ↳ partial/unreachable —>
reintento cada 10 min
revoke() —> revoked/pending —> DELETE a la VPS —> revoked/synced

```

Las reglas que no son obvias, todas en [credential.service.ts](#):

- **issue()** es idempotente por (sujeto, ámbito): si ya hay credencial viva, la devuelve.
- **reschedule()** y **changeScope()** no tocan el PIN. El dueño ya lo tiene anotado; cambiarlo por un cambio de fechas lo dejaría fuera.
- **pin_conflict** regenera y reemite, hasta 3 veces. El terminal comparte espacio de PINes con residentes cargados a mano que el backend no conoce, así que el índice único de Postgres no basta.
- **La llamada a la VPS queda fuera de toda transacción.** Si el proceso muere ahí, la fila ya está en **pending** y el barrido la recoge.
- **partial no es un error:** el PIN escrito en el portal peatonal pero no en la barrera deja entrar a pie, y el panel debe poder decirlo.

5. De dónde salen las credenciales

Automático — **POST /webhooks/hostaway**. Responde **200** siempre y trabaja en segundo plano: un fallo de accesos no puede romper nada aguas arriba. Protegido por **HOSTAWAY_WEBHOOK_SECRET** con **timingSafeEqual**; sin secreto configurado responde **503** en vez de quedar abierto.

El identificador es **hostawayReservationId (o **id**), nunca **reservationId**.** La respuesta de Hostaway trae los tres: los dos primeros son el id interno ("**65895170**"), y **reservationId** / **channelReservationId** son el del canal (**563484-guest-...-HMFQM523QX**). Como el barrido usa el interno, tomar el del canal daría **dos credenciales vivas para la misma reserva**, y el índice único no puede verlo porque los **subject_ref** difieren.

Solo emiten **new, **modified**, **confirmed** y **ownerStay**** — lista blanca, no negra. Con lista negra, un **inquiry** (una consulta de alguien interesado, que ni siquiera es una reserva) abría la puerta del edificio. Los estados de cancelación revocan; el resto no proyecta la estancia siquiera. [guest-stay.service.ts](#) resuelve el edificio por **Unit.HostawayListingID** en openMAINT, calcula la vigencia y emite. Reserva cancelada (**cancelled** / **declined** / **expired**) revoca en bloque.

Emite **siempre **pedestrian****: no hay ninguna fuente que diga si el huésped trae vehículo. Ampliarlo es manual, con **POST /access/credentials/:id/scope**.

Manual — **POST /access/credentials**. Para residentes y personal, y para cualquier caso que el webhook no cubra.

Vigencia. Manda la hora de la propia reserva (**checkInTime** / **checkOutTime**), que Hostaway da **por reserva y no es igual en todos los listings** — hay listings de Pradera con **checkInTime: 16** frente al 15 habitual. **ACCESS_CHECKIN_HOUR** / **ACCESS_CHECKOUT_HOUR** son solo el respaldo cuando la reserva no las trae. A eso se le restan **ACCESS_GUEST_LEAD_HOURS** y se le suman **ACCESS_GUEST_GRACE_HOURS**. La conversión a instante ocurre en **un único punto** (**LOCAL_UTC_OFFSET = '-05:00'**, Ecuador no tiene horario de verano).

6. Endpoints

Todos bajo sesión de openMAINT (`x-session-token`) y rol `SuperUser` — no existe un rol con code `Admin`.

Ruta	Nota
<code>GET /access/credentials</code>	Filtra por <code>subject</code> , <code>status</code> , <code>buildingId</code> . Nunca devuelve el PIN : expone <code>pinConfigured</code> booleano
<code>POST /access/credentials</code>	Alta manual. <code>400</code> si el edificio no tiene cobertura
<code>POST /access/credentials/:id/revoke</code>	Con motivo
<code>POST /access/credentials/:id/scope</code>	Asignación manual de ámbito vehicular. El PIN no cambia
<code>GET /access/credentials/:id/pin</code>	Excepción temporal . Ver abajo
<code>GET /access/health</code>	Estado por edificio + credenciales en <code>failed</code>
<code>POST /webhooks/hostaway</code>	Sin sesión: secreto compartido en cabecera

`GET /access/credentials/:id/pin` es deliberadamente una excepción. Sin portal no hay canal de entrega y soporte necesita poder leer el PIN. Está detrás de `ACCESS_ALLOW_PIN_REVEAL` (`false` por defecto), exige rol de administración y deja `logger.warn` con credencial y usuario en cada lectura. Se apaga el día que el portal entre en servicio.

7. Barridos

@Cron con `BUSINESS_TIMEZONE`, todos con el mismo interruptor `ACCESS_SCHEDULER_ENABLED` comprobado dentro del método: los decoradores se evalúan al importar, antes de que Nest instancie nada.

Cuándo	Qué hace
<code>* / 10 * * *</code> <code>*</code>	Reintenta lo no sincronizado. Agotado <code>ACCESS_SYNC_MAX_ATTEMPTS</code> , marca <code>failed</code>
<code>10 * * * *</code>	Marca <code>expired</code> lo vencido. Contabilidad: el terminal ya lo ignora solo
<code>0 3 * * *</code>	Purga: <code>DELETE</code> de lo <code>expired/revoked</code> aún escrito. Sin esto el terminal se llena y rechaza altas
<code>30 3 * * *</code>	Concilia contra el inventario real de cada dispositivo
<code>0 5 * * *</code>	Repasa 14 días de llegadas en Hostaway. Red de seguridad del webhook

La conciliación nunca borra lo que no lleva prefijo `DT4-`: lo reporta y lo deja. Un barrido «limpiador» dejaría a residentes fuera de su casa. Y el inventario **se pagina siempre**; un tope fijo trunca en silencio en terminales llenos.

El barrido de reservas existe porque un webhook perdido no se nota: la reserva no se proyecta y el huésped llega sin PIN, o peor, una cancelación no entregada deja un PIN vivo.

Usa `getReservationsForAccess()`, no la consulta de facturación. Aquella filtra por `paymentStatus === 'Paid'`, y una reserva directa llega con `'Unknown'`: el barrido no la veía y, como interpreta «ausente de

Hostaway» = «cancelada», le revocaba el PIN a un huésped legítimo. La consulta de accesos no filtra por estado ni por cobro — quién emite y quién revoca se decide en [GuestStayService](#).

8. Variables de entorno

Definidas en [.env.example](#).

Variable	
<code>ACCESS_PIN_KEY</code>	Obligatoria. AES-256-GCM, 32 bytes en base64. Rotarla exige re-cifrar la tabla
<code>ACCESS_PIN_FINGERPRINT_KEY</code>	Obligatoria. HMAC de la huella. Distinta de la anterior, para que comprometer la huella no ayude a descifrar
<code>ACCESS_IOT_URL</code>	Obligatoria. VPS central, sin barra final
<code>ACCESS_IOT_TOKEN</code>	Obligatoria. Service token de Cloudflare Access, formato <code><client-id>:<client-secret></code>
<code>HOSTAWAY_WEBHOOK_SECRET</code>	Obligatoria. Vacío ⇒ el webhook responde <code>503</code>
<code>ACCESS_IOT_USE MOCK</code>	<code>true</code> responde desde el mock sin salir a la red. Obligatorio en E2E
<code>ACCESS_PIN_LENGTH</code>	4 por decisión del cliente
<code>ACCESS_PIN_COOLDOWN_DAYS</code>	30. Días antes de reasignar un PIN liberado
<code>ACCESS_SYNC_MAX_ATTEMPTS</code>	5
<code>ACCESS_ALLOW_PIN_REVEAL</code>	<code>false</code>
<code>ACCESS_SCHEDULER_ENABLED</code>	<code>false</code> . Solo <code>true</code> activa los barridos
<code>ACCESS_CHECKIN_HOUR / ACCESS_CHECKOUT_HOUR</code>	15 / 11. Solo respaldo: manda el <code>checkInTime</code> de la reserva
<code>ACCESS_GUEST_LEAD_HOURS / ACCESS_GUEST_GRACE_HOURS</code>	Margen antes de llegada y después de salida

Las dos claves se generan igual, y **no son intercambiables**:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

9. Pruebas

```
docker compose up -d && npm run migration:run # Postgres local en :5555
npm test # unitarias
npm run test:e2e # suite access-control
```

Las E2E fijan sus variables en `test/setup-env.ts` con `=` literal, no `??=`: el `ConfigModule` carga el `.env` del directorio de trabajo, y sin fijarlas la suite dependía de la configuración local de quien la ejecutara.

Para probar a mano el flujo completo, `ACCESS_IOT_USE MOCK=true` y emitir por `POST /access/credentials`. El mock tiene escenarios deterministas: `PRA-VEHICULAR-1` está caído a propósito (camino `partial`) y `ING-PEATONAL-1` tiene un usuario manual con PIN `4821` (camino `pin_conflict` y usuario ajeno para la conciliación).

10. Lo que aún no existe

- **Portal del huésped.** Confirmado, sprint posterior. Faltan `GuestTokenService` con HMAC propio, guard de sesión y entrega del enlace. En un edificio sin PIN se oculta el bloque de credenciales y el resto del portal se conserva.
- **Auditoría de aperturas.** Nada lee `AcsEvent` hoy, ni aquí ni en la VPS. La tabla `access_event` vuelve cuando exista la fuente.
- **Sensores ambientales.** El «estado de la habitación» del portal se refiere a ellos. No existen.